MAM4 - Programmation C++

Correction de l'examen 2024-2025

Exercice : une zoologie de classes

Le sujet demande de corriger les bugs du code du répertoire `Exercice` (classe abstraite `Animal` avec `Deplacement()` et `Cri()`). Ce code n'est pas dans le PDF que j'ai reçu, donc je ne peux pas dire quelle ligne est fausse.

Voici la liste des bugs classiques sur ce genre d'exercice, à vérifier un par un dans le code :

- méthode non déclarée `virtual` dans `Animal` : pas de polymorphisme, on appelle toujours la version de la classe mère ;
- classe pas vraiment abstraite : il manque le `= 0` sur au moins une méthode ;
- signature différente dans la classe fille (`const` oublié, type de retour ou argument différent) : la méthode cache l'autre au lieu de la redéfinir ;
- destructeur de `Animal` non virtuel : le `delete` via un pointeur `Animal*` détruit mal l'objet ;
- héritage privé (`class Chien : Animal`) au lieu de `public` ;
- attribut `private` utilisé dans une classe fille (il faut `protected`) ;
- objets stockés par valeur dans un tableau d'`Animal` (slicing) au lieu de pointeurs ;
- oubli du `;` après l'accolade de la classe, des gardes d'inclusion ou d'un `#include`.

Si tu m'envoies le dossier Exercice, je fais la correction ligne par ligne (ligne fausse, ligne corrigée, commentaire `//bug`) et je l'ajoute au PDF.

Problème : maillage pour les éléments finis

Partie 1 : la classe Node

Un noeud est caractérisé par ses coordonnées (x, y). Il reste à écrire cinq fonctions membres : 3 constructeurs, 2 accesseurs avec leurs versions `const`, et la surcharge de <<. Je n'ai pas le fichier `Node.hpp` du sujet, donc les signatures ci-dessous sont celles que j'ai choisies : adapte-les si les tiennes sont différentes.

Node.hpp

```
#ifndef NODE_HPP
#define NODE_HPP

#include <iostream>

// Un noeud du maillage : simplement ses coordonnees (x, y)
class Node {
private:
    double x_, y_;

public:
    // 3 constructeurs
    Node();                          // par defaut : (0, 0)
    Node(double x, double y);        // avec les coordonnees
    Node(const Node& n);             // par copie

    // operateur d'affectation (necessaire car on a ecrit le constructeur par copie)
    Node& operator=(const Node& n);

    // 2 accesseurs + leurs versions const
    double& get_x();
    double& get_y();
    const double& get_x() const;
    const double& get_y() const;
};

// surcharge de l'operateur de flux
std::ostream& operator<<(std::ostream& out, const Node& n);

#endif
```

Node.cpp

```
#include "Node.hpp"

Node::Node() : x_(0.), y_(0.) {}

Node::Node(double x, double y) : x_(x), y_(y) {}

Node::Node(const Node& n) : x_(n.x_), y_(n.y_) {}

Node& Node::operator=(const Node& n) {
    x_ = n.x_;
    y_ = n.y_;
    return *this;
}

double& Node::get_x() { return x_; }
double& Node::get_y() { return y_; }

const double& Node::get_x() const { return x_; }
const double& Node::get_y() const { return y_; }

std::ostream& operator<<(std::ostream& out, const Node& n) {
    out << "(" << n.get_x() << ", " << n.get_y() << ")";
    return out;
}
```

Ce qu'il faut retenir

- **3 constructeurs** : par défaut (0, 0), avec (x, y), par copie. On utilise la liste d'initialisation `: x_(x), y_(y)`.
- **Pourquoi 2 versions de chaque accesseur ?** La version non const renvoie une référence modifiable (`a.get_x() = 3.`). La version const est appelée sur un objet `const` (par exemple dans `operator<<` où le noeud est passé en `const Node&`) : lecture seule.
- **operator<< n'est pas une fonction membre** : l'opérande de gauche est le flux (`std::ostream`), pas un Node. Elle renvoie le flux par référence pour pouvoir enchainer les <<.
- **Règle des trois** : dès qu'on écrit le constructeur par copie, on écrit aussi l'opérateur = (le compilateur le signale avec -Wall -Wextra). Il sera utilisé plus loin dans triangle et quadrangle.

test_node.cpp

```
#include <iostream>
#include "Node.hpp"

int main() {
    Node a;               // constructeur par defaut
    Node b(1.5, 2.);      // constructeur (x, y)
    Node c(b);            // constructeur par copie

    std::cout << "a = " << a << std::endl;   // (0, 0)
    std::cout << "b = " << b << std::endl;   // (1.5, 2)
    std::cout << "c = " << c << std::endl;   // (1.5, 2)

    // accesseur non const : on peut modifier
    a.get_x() = 3.;
    a.get_y() = 4.;
    std::cout << "a modifie = " << a << std::endl;   // (3, 4)

    // accesseur const : lecture seule
    const Node d(5., 6.);
    std::cout << "d.x = " << d.get_x() << ", d.y = " << d.get_y() << std::endl;

    return 0;
}
```

Sortie obtenue (compilé avec `g++ -Wall -Wextra`, aucun warning) :

```
a = (0, 0)
b = (1.5, 2)
c = (1.5, 2)
a modifie = (3, 4)
d.x = 5, d.y = 6
```

Partie 2 : liste d'éléments (polymorphisme)

On veut un tableau de cellules qui peuvent être des triangles ou des quadrangles. On utilise une classe mère abstraite et deux classes filles.

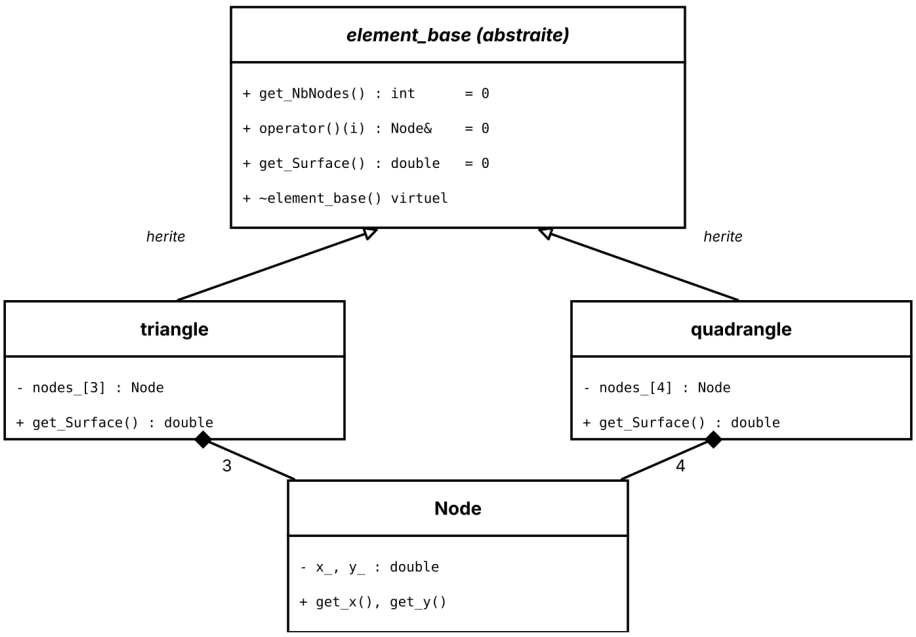


Figure A - Diagramme UML (flèche vide = héritage, losange plein = composition).

Questions 1 à 4 : Element.hpp

- **Q1, element_base** : les trois méthodes sont *virtuelles pures* (`= 0`), donc on ne peut pas créer d'objet `element_base`. Le **destructeur est virtuel** : sans ça, `delete` sur un `element_base*` ne détruirait pas correctement l'objet réel.
- **Q2, triangle** : tableau de 3 noeuds, constructeur qui prend les 3 noeuds en argument.
- **Q3, quadrangle** : tableau de 4 noeuds, constructeur qui prend les 4 noeuds.
- **Q4, aire** : fonction membre `get_Surface()` dans chaque classe (la méthode virtuelle pure de la classe mère).

Aire du triangle ABC (produit vectoriel) :

$$S = 1/2 \ | (x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A) |$$

Aire du quadrangle ABCD (noeuds donnés dans l'ordre autour de la cellule) : on le coupe en deux triangles ABC et ACD et on additionne. Cela marche pour un quadrangle convexe (cas de ce maillage).

Element.hpp

```
#ifndef ELEMENT_HPP
#define ELEMENT_HPP

#include <cmath>
#include "Node.hpp"

// ---------- Q1 : classe abstraite ----------
class element_base {
public:
    virtual ~element_base() {}                                  // destructeur virtuel
    virtual int get_NbNodes() const = 0;                        // nombre de noeuds
    virtual const Node& operator()(int i) const = 0;            // i-eme noeud
    virtual double get_Surface() const = 0;                     // aire (Q4)
};

// ---------- Q2 : triangle ----------
class triangle : public element_base {
private:
    Node nodes_[3];

public:
    triangle(const Node& a, const Node& b, const Node& c) {
        nodes_[0] = a;
        nodes_[1] = b;
        nodes_[2] = c;
    }

    int get_NbNodes() const { return 3; }

    const Node& operator()(int i) const { return nodes_[i]; }

    // aire = 1/2 |(xB-xA)(yC-yA) - (xC-xA)(yB-yA)|
    double get_Surface() const {
        double ax = nodes_[0].get_x(), ay = nodes_[0].get_y();
        double bx = nodes_[1].get_x(), by = nodes_[1].get_y();
        double cx = nodes_[2].get_x(), cy = nodes_[2].get_y();
        return 0.5 * std::fabs((bx - ax) * (cy - ay) - (cx - ax) * (by - ay));
    }
};

// ---------- Q3 : quadrangle ----------
class quadrangle : public element_base {
private:
    Node nodes_[4];

public:
    quadrangle(const Node& a, const Node& b, const Node& c, const Node& d) {
        nodes_[0] = a;
        nodes_[1] = b;
        nodes_[2] = c;
        nodes_[3] = d;
    }

    int get_NbNodes() const { return 4; }

    const Node& operator()(int i) const { return nodes_[i]; }

    // aire = somme des aires des triangles (A,B,C) et (A,C,D)
    // (les noeuds sont donnes dans l'ordre autour du quadrangle)
    double get_Surface() const {
        triangle t1(nodes_[0], nodes_[1], nodes_[2]);
        triangle t2(nodes_[0], nodes_[2], nodes_[3]);
        return t1.get_Surface() + t2.get_Surface();
    }
};

#endif
```

Question 5 : le maillage de la figure 1 et tu_mesh.cpp

Le sujet écrit « figure 1 gauche » à la question 5, mais l'introduction de cette partie parle du maillage de la figure 1 **droite** (le maillage hybride triangles + quadrangles). Le maillage de gauche n'a que des triangles : j'ai donc traité celui de droite.

Lecture de la figure : les noeuds sont placés sur une grille 3x3 de pas 1, avec la case en bas à gauche vide. La numérotation est celle de la figure (le noeud central porte le numéro 1, la diagonale va du noeud 3 au noeud 1).

Noeud	0	1	2	3	4	5	6	7
(x, y)	(0,1)	(1,1)	(1,2)	(0,2)	(2,1)	(2,2)	(1,0)	(2,0)

Cellule	Type	Noeuds	Aire
0	triangle	0, 1, 3	0.5
1	triangle	3, 1, 2	0.5
2	quadrangle	1, 4, 5, 2	1
3	quadrangle	6, 7, 4, 1	1
		Total	**3**

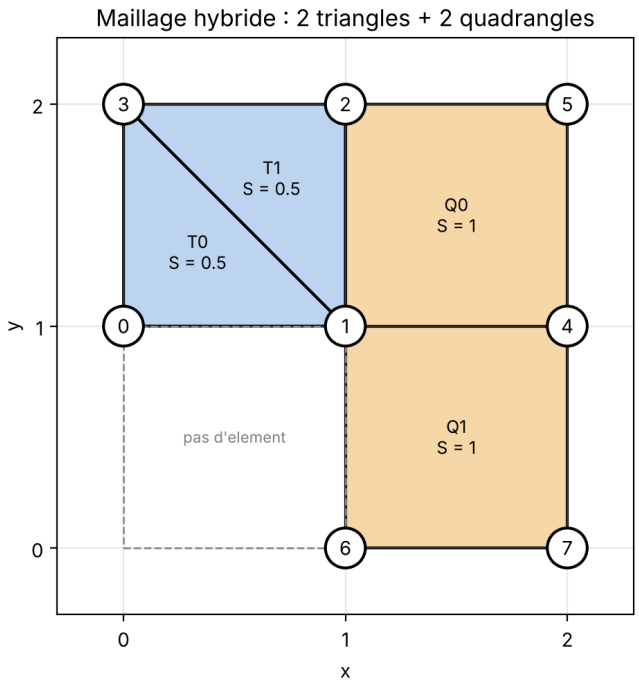


Figure B - Le maillage hybride, avec les aires calculées.

tu_mesh.cpp

```
#include <iostream>
#include <vector>
#include "Node.hpp"
#include "Element.hpp"

// Q5 : maillage hybride de la figure 1 (a droite)
void test() {
    // les 8 noeuds, numerotes comme sur la figure
    Node n[8];
    n[0] = Node(0., 1.);
    n[1] = Node(1., 1.);   // noeud central
    n[2] = Node(1., 2.);
    n[3] = Node(0., 2.);
    n[4] = Node(2., 1.);
    n[5] = Node(2., 2.);
    n[6] = Node(1., 0.);
    n[7] = Node(2., 0.);

    // le maillage : tableau de cellules (triangles ou quadrangles)
    std::vector<element_base*> mesh;
    mesh.push_back(new triangle(n[0], n[1], n[3]));
    mesh.push_back(new triangle(n[3], n[1], n[2]));
    mesh.push_back(new quadrangle(n[1], n[4], n[5], n[2]));
    mesh.push_back(new quadrangle(n[6], n[7], n[4], n[1]));

    // liste des surfaces (appel polymorphe)
    double total = 0.;
    for (int i = 0; i < (int)mesh.size(); i++) {
        std::cout << "element " << i << " : " << mesh[i]->get_NbNodes()
                  << " noeuds, surface = " << mesh[i]->get_Surface() << std::endl;
        total += mesh[i]->get_Surface();
    }
    std::cout << "surface totale = " << total << std::endl;

    // liberation de la memoire
    for (int i = 0; i < (int)mesh.size(); i++)
        delete mesh[i];
}

int main() {
    test();
    return 0;
}
```

Le tableau `std::vector<element_base*>` contient indifféremment des triangles et des quadrangles. L'appel `mesh[i]->get_Surface()` choisit la bonne version à l'exécution (polymorphisme). Les objets créés avec `new` sont libérés avec `delete` à la fin.

Sortie obtenue :

```
element 0 : 3 noeuds, surface = 0.5
element 1 : 3 noeuds, surface = 0.5
element 2 : 4 noeuds, surface = 1
element 3 : 4 noeuds, surface = 1
surface totale = 3
```

Les aires correspondent à ce qu'on voit sur la figure : deux demi-carrés (0.5 chacun) et deux carrés unité (1 chacun), soit 3 au total.

Compilation

```
g++ -Wall -Wextra -o test_node test_node.cpp Node.cpp
g++ -Wall -Wextra -o tu_mesh   tu_mesh.cpp   Node.cpp
```